



ATIVIDADE 01 - AIRFLOW

1. Contexto:

*Você acaba de assumir como líder técnico (tech lead) do time de dados da **ShopBrasil**, um marketplace de e-commerce em rápido crescimento.*

*Todas as manhãs, antes que a equipe de pricing e os gerentes de categoria comecem o expediente, eles **abrem um painel com o panorama de preços por categoria, preço médio, mínimo, máximo e quantidade de produtos**, para decidir as promoções e reposições do dia. Atualmente esse painel é alimentado por um **script Python**, agendado via **cron**, que busca os dados de uma API de catálogo.*

1.1. Situação Problema:

A arquitetura atual é frágil, pois, quando a API oscila de madrugada, o script falha silenciosamente e o time fica sem dados. Quando alguém o roda "**na mão**" de novamente, os números aparecem duplicados na base de dados, e a cada nova categoria que surge, é preciso editar o código linha a linha. Com isso temos uma arquitetura que **não escala, não é confiável e ninguém dorme tranquilo**.

2. Objetivo

Sua missão é projetar e conduzir seu time na construção de um pipeline de verdade no **Apache Airflow**, que aposente esse script executado via cron.

O pipeline deve coletar os produtos da API de catálogo (representada aqui pela FakeStore API), calcular as métricas por categoria e gravar o resultado na base analítica em PostgreSQL. Como tech lead, você não só implementa a referência, mas, define os padrões que o time vai seguir.

Na prática, o pipeline precisa:

- rodar sozinho todo dia às 06:00 (horário de Brasília), antes de o time chegar;
- resistir às instabilidades da API, retentando com calma e sem derrubar a execução inteira;
- escalar sozinho à medida que novas categorias aparecem;
- nunca duplicar dados quando for reprocessado;
- avisar quando algo falhar;
- e ser modular e legível, para que qualquer pessoa do time consiga manter.



3. Requisitos obrigatórios

Fonte de Dados:

- API: <https://fakestoreapi.com/docs>
 - Esse é o Swagger da API, da qual vocês irão capturar os dados.

Modelagem e estrutura:

- Usar TaskFlow API (@dag / @task); dependências saem da chamada das funções.
- XComs automáticos via return (passar apenas dados pequenos entre tasks).
- O pipeline deve conter, de forma identificável, as topologias linear, fan-out (mapeamento por categoria) e fan-in (consolidação).

Agendamento:

- Timezone ancorado em America/Sao_Paulo (use pendulum), com start_date e catchup=False.

Ingestão resiliente:

- Task “Buscar Produtos” com retry e exponential backoff
- Tratamento de erro com try/except + raise (para acionar o retry).
- Callbacks de ciclo de vida na task crítica: *on_failure_callback*, *on_retry_callback* e *on_success_callback*.

Processamento paralelo

- Calcular métricas por categoria com *Dynamic Task Mapping* (.expand(...)).
- **Pool** para limitar a concorrência das tasks mapeadas (pool ecommerce_pool com 2 slots).

Organização

Agrupar as tasks em pelo menos 2 TaskGroups (ex.: ingestão e análise).

Persistência no PostgreSQL

- Criar Task que:
 - Salve os registros no Banco de Dados com consistência na inserção de dados para não gerar duplicidade de registros.
 - use PostgresHook e uma Connection do Airflow.
- A gravação deve ser **idempotente**: re-rodar a mesma run não duplica linhas.





4. Requisitos Opcionais

Operador customizado

- *ValidarProdutosOperator(BaseOperator)* para validar o schema dos produtos antes do processamento.

Tabela de histórico

- Além do snapshot *idempotente*, gravar uma 2ª tabela em ***modo append*** com a data da execução
 - *A ideia é acompanhar a evolução dos preços.*

SLA / alerta

- Configurar ***SLA*** ou um ***on_failure_callback*** que simule envio de alerta.

5. Modo de Entrega

- Disponibilizar o projeto em um repositório (github, gitlab, etc).
- Liberar acesso de visualização do repositório
- Todo o pipeline deve estar contido em um projeto dockerizado.
- Postar o link do projeto.